As an Executive Director at Nomura, I evaluate quantitative researchers not just on their ability to write models, but on their ability to ensure those models **train stably**. In the Central Risk Book (CRB), we often utilize deep neural architectures for market impact prediction and alpha signal refinement. If your initialization strategy is flawed, your training will either diverge or vanish, rendering your research pipeline useless.

What follows is a painstaking, first-principles derivation of why naive initialization fails and how variance-preservation initialization schemes maintain signal integrity across deep layers.

---

1. The Symmetry-Breaking Argument: First Principles

If we initialize all weights $W_{ij}^{(l)}$ in a layer l to the same constant c (e.g., $c = 0$), we effectively destroy the capacity of the neural network.

The Mathematical Proof

Consider a neuron j in layer l. The pre-activation $a_j^{(l)}$ is the dot product of the weights and the previous layer's activations $z^{(l-1)}$:

$$a_j^{(l)} = \sum_{i=1}^{n_{l-1}} W_{ij}^{(l)} z_i^{(l-1)} + b_j^{(l)}$$

If $W_{ij}^{(l)} = c$ for all i, j, then:

$$a_j^{(l)} = c \sum_{i=1}^{n_{l-1}} z_i^{(l-1)} + b_j^{(l)}$$

Since this summation is identical for every neuron j in layer l, it follows that $a_1^{(l)} = a_2^{(l)} = \cdots = a_{n_l}^{(l)}$.
Because all neurons receive identical inputs and have identical weights, their gradients will also be identical:

$$\frac{\partial \mathcal{L}}{\partial W_{1j}} = \frac{\partial \mathcal{L}}{\partial W_{2j}}$$

During backpropagation, these weights will be updated by the same delta. They will remain equal throughout the entire training process. A layer with 100 neurons becomes mathematically equivalent to a layer with **one single neuron**. The network loses its ability to learn complex feature representations.

---

2. Variance Preservation: Forward and Backward Passes

To avoid vanishing or exploding gradients, we want the variance of the activations to remain constant across layers:

$$\mathrm{Var}(z^{(l)}) = \mathrm{Var}(z^{(l-1)})$$

Derivation of the Variance Constraint

For a layer l with n_l inputs and n_{l+1} outputs:

1. Assume weights W have mean 0 and variance σ_W^2.

2. Assume inputs z are independent and have variance $\text{Var}(z)$.
3. The variance of the pre-activation $a^{(l)}$ is:

$$\text{Var}(a^{(l)}) = n_l \cdot \text{Var}(W \cdot z) = n_l \cdot \text{Var}(W) \cdot \text{Var}(z)$$

To keep $\text{Var}(a^{(l)}) = \text{Var}(z)$, we require:

$$n_l \cdot \text{Var}(W) = 1 \implies \text{Var}(W) = \frac{1}{n_l}$$

The Xavier (Glorot) Compromise

In deep networks, we must consider both the **forward pass** (signal propagation) and the **backward pass** (gradient propagation). The forward pass requires $\text{Var}(W) = 1/n_l$, while the backward pass (where the gradient flows through n_{l+1} neurons) requires $\text{Var}(W) = 1/n_{l+1}$.

To balance these, Xavier initialization uses the arithmetic mean of the fan-in and fan-out:

$$\text{Var}(W) = \frac{2}{n_l + n_{l+1}}$$

If we draw weights from a uniform distribution $\mathcal{U}(-a, a)$, where $\text{Var}(\mathcal{U}) = \frac{(2a)^2}{12}$, setting this equal to $\frac{2}{n_l+n_{l+1}}$ yields:

$$a = \sqrt{\frac{6}{n_l + n_{l+1}}}$$

---

3. He Initialization: Adjusting for ReLU

The Xavier derivation assumes a linear activation or a symmetric non-linearity (like $\tanh$). However, if we use **ReLU** ($f(x) = \max(0, x)$), we encounter a new problem:

1. ReLU sets all negative inputs to zero.
2. Statistically, for a centered distribution, half of the inputs are killed.
3. Therefore, the variance of the activation is halved:

$$\text{Var}(z^{(l)}) = \frac{1}{2} n_l \text{Var}(W) \text{Var}(z^{(l-1)})$$

To maintain unit variance, we must double the variance of our weights:

$$\text{Var}(W) = \frac{2}{n_l}$$

This is the **He Initialization**. It accounts for the fact that ReLU units do not pass signal through the negative half of the input space, requiring larger weight magnitudes to compensate for the "lost" variance.

---

4. Implementation Logic

When implementing this in your code for the Nomura CRB pipeline, do not hard-code constants. Use the fan-in (n_{in}) and fan-out (n_{out}) dynamically:

```python
import numpy as np

def initialize_weights(n_in, n_out, method='he'):
    """
    Dynamically initializes weights to prevent signal degradation.
    """
    if method == 'xavier':
        # Var = 2 / (n_in + n_out)
        std = np.sqrt(2.0 / (n_in + n_out))
    elif method == 'he':
        # Var = 2 / n_in
        std = np.sqrt(2.0 / n_in)
    else:
        raise ValueError("Method not supported")

    return np.random.normal(0, std, size=(n_in, n_out))
```

Why this matters for the CRB desk:

In our market impact models, we often use deep networks to capture highly non-linear, multi-scale dependencies (e.g., order flow imbalances at different time horizons). If you initialize naively, you will find that early layers have small weights (no learning) and later layers have massive weights (gradient explosion). By using **He initialization**, we ensure that even with 50+ layers, the signal variance of our market-state features remains stable, allowing for effective convergence of our execution policies.